SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

CSA37- SOFTWARE TESTING

EXPERIMENT 3

Case Study: ARMS Portal – Negative Test Scenarios and Test Cases

AIM:

To identify and design negative test scenarios and test cases for the ARMS Portal to verify its behavior under invalid inputs, incomplete data, unauthorized access and expired sessions, and to identify and report software defects.

Problem Description

An ARMS Portal allows students to log in, view their academic profile, check attendance and marks, access academic information, submit permitted requests, and view notifications. Students must provide valid credentials and the portal must validate mandatory fields and access permissions before processing requests. The testing team is responsible for verifying whether the application works correctly under valid and invalid conditions. They must prepare positive and negative test cases and document any defects identified during testing.

Functional Requirements

1. Users must log in using valid credentials.

2. Students must be able to view correct profile information.

3. Attendance and marks must be displayed accurately.

4. Only permitted academic requests can be submitted.

5. The portal must display appropriate error messages for invalid inputs.

3. Negative Test Cases

Negative testing verifies how the application behaves when invalid, incomplete, or unexpected inputs are provided.

Test Case ID	Test Scenario	Test Input / Action	Expected Result	Status
NT01	Login with wrong password	Correct username and incorrect password	Login denied with error message	Pass
NT02	Submit blank login	Leave username/password blank	Required-field messages displayed	Pass
NT03	Enter invalid student ID	Enter invalid/nonexistent ID	Invalid ID rejected	Pass
NT04	Submit incomplete request	Leave mandatory request field blank	Validation message displayed	Pass
NT05	Access restricted module	Attempt unauthorized module	Access denied	Pass
NT06	Use expired session	Open protected page after session expiry	User redirected to login	Fail

NT07	Enter invalid date/value	Use invalid format in request	Invalid input rejected	Pass
NT08	Submit duplicate request	Submit same request twice	Duplicate request prevented or warning shown	Pass

4. Detailed Execution of a Failed Test Case

Test Case NT06
Failed

Test scenario: Use expired session

Precondition

The user is logged out or session has expired.

Test steps

1. Log in.
2. Open a protected portal page.
3. Wait for session expiry.
4. Use the browser Back button or direct URL.
5. Observe the page.

Expected result

The portal should redirect the user to the login page and deny access.

Actual result

The protected page remains accessible after session expiry.

Conclusion

The test fails because expired sessions are not enforced correctly.

5. Sample Bug Report

A bug report is a document that records a defect discovered during software testing. It provides developers with sufficient information to reproduce, investigate, and fix the issue.

Field	Details
Bug ID	BR-001
Linked Test Case	NT06
Severity	High
Priority	High
Bug Title	Use expired session
Environment	Application: ARMS Portal; Browser: Google Chrome; Operating System: Windows 10; Test Module: Relevant application module
Preconditions	The user is logged out or session has expired.
Steps to Reproduce	1. Log in. 2. Open a protected portal page. 3. Wait for session expiry. 4. Use the browser Back button or direct URL. 5. Observe the page.
Expected Result	The portal should redirect the user to the login page and deny access.

Actual Result	The protected page remains accessible after session expiry.
Impact	The defect may cause incorrect system behavior, data inconsistency, transaction errors, or poor user experience.
Suggested Fix	Apply validation on the client and server side and repeat the validation at the final submission/transaction stage.
Status	Open — awaiting developer investigation and correction.

6. Bug Life Cycle

New — Tester reports the defect
Assigned — Developer receives the defect
In Progress — Developer investigates and fixes it
Fixed — Developer submits the correction
Retesting — Tester verifies the correction
Closed — Tester confirms the defect is resolved
If the defect still exists after the fix, the tester reopens the bug and sends it back for further correction.

7. Difference Between Positive and Negative Testing

Positive Testing	**Negative Testing**
Uses valid inputs.	Uses invalid or unexpected inputs.
Checks whether expected operations succeed.	Checks whether errors are handled correctly.
Example: valid login or valid transaction.	Example: wrong password or invalid transaction.
Expected result is successful execution.	Expected result is safe rejection or appropriate error handling.